Brasil Hub

Documento de Arquitetura

Versão 2.0

Tabela 1 - Integrantes do Grupo

Matrícula	Nome	Função (responsabilidade)	Pontos de participação na elaboração
251034885	Ágatha Natasha Ferreira da Silva Vieira	Desenvolvedor Back-end e Infraestrutura (Âncora)	12,5
251035031	César Augusto Pereira Silva	Desenvolvedor Front-end (Âncora)	12,5
251020226	Eduardo de Sousa Brito	Desenvolvedor Back-end e Infraestrutura	12,5
251008526	Fabiana Reis Souza	Product Owner e Desenvolvedor Front-end (Âncora)	12,5
251008535	Felipe George Carvalho Dourado de Aragão	Desenvolvedor Front-end (Rotativo)	12,5
251023282	Josef Wojtyla Barros de Souza	Scrum Master / Desenvolvedor Back-end e Infraestrutura (Âncora)	12,5
251012304	Lucas Abdalla Nery	Desenvolvedor Front-end (Rotativo)	12,5

242015325	Paulo Eduardo Fidelis Martins	Desenvolvedor Back-end e Infraestrutura (Rotativo)	12,5
Total de pontos (Devem fechar 100 pontos para toda a equipe.)			100

Tabela 2 - Histórico de Revisões

Data	Versão	Descrição	Autor(es)
03/10/2026	1.0	Elaboração Inicial do Documento de Arquitetura	Todos
16/10/2026	1.1	Atualização de tecnologias	Josef
09/11/2026	2.0	Correções de textos e alinhamento com documento de visão	Josef

Sumário

1 Introdução *5*
1.1 Propósito **5**
1.2 Escopo **5**
2 Representação Arquitetural *6*
2.1 Definições **6**
2.2 Justificativa da escolha **7**
2.3 Detalhamento **8**
2.3.1 Descrição do Diagrama Arquitetural **8**
2.3.2 Modelo de persistência **11**
2.4 Metas e restrições arquiteturais **12**
2.5 Visões **13**
2.5.1 Visão uso **13**
2.5.2 Visão de organização lógica **16**
2.5.2.1 Razão lógica de cada módulo: **17**
2.5.2.2 Comunicação entre os módulos: **19**
2.5.2.3 Fluxos principais **19**
2.5.3 Visão estrutural **22**
2.6 Visão de Implantação **24**
2.7 Restrições adicionais **25**
3 Bibliografia *26*

1 Introdução

1.1 Propósito

Este documento descreve a arquitetura do sistema Brasil Hub, desenvolvido na disciplina de Métodos de Desenvolvimento de Software (MDS), no segundo semestre de 2026, tendo como base os direcionadores definidos no Documento de Visão e Projeto v1.1 (GRUPO DIJKSTRA, 2026). Seu propósito é apresentar, de forma técnica e estruturada, as principais decisões arquiteturais adotadas pela equipe, os componentes que constituem o sistema, suas responsabilidades, as interfaces de comunicação e a infraestrutura necessária para sua execução.

O documento tem como finalidade orientar o desenvolvimento, a integração e a manutenção do software, fornecendo uma referência comum para os integrantes da equipe, bem como para professores, monitores e demais interessados na solução. São apresentados o estilo arquitetural adotado, a organização lógica dos módulos, a estrutura dos componentes, as metas e restrições arquiteturais, as visões do sistema e o ambiente de implantação.

1.2 Escopo

O escopo detalhado do produto está definido no Documento de Visão e Projeto v1.1 (GRUPO DIJKSTRA, 2026), em especial nas Tabelas 10 a 13, que estabelecem o backlog de requisitos funcionais e não funcionais priorizados pelo método MoSCoW.

As funcionalidades e restrições arquiteturalmente relevantes correspondem diretamente aos requisitos classificados como **MUST** nas Tabelas 10 a 13 do Documento de Visão (GRUPO DIJKSTRA, 2026):

- **B-01 (Mapa Interativo do Brasil):** visualização espacial dos indicadores socioeconômicos por Unidade da Federação;
- **B-02 (Filtros Dinâmicos):** seleção e consulta combinada por estado, dimensões (renda, educação, longevidade) e período;
- **B-03 (Recursos de Acessibilidade):** conformidade de interface com suporte a redimensionamento de fontes, alto contraste e modo noturno;

- **B-04 (Administração e Upload de Dados):** upload de arquivos CSV pelo administrador, com validação de formato e tratamento prévio à persistência;
- **B-05 (Desempenho da API):** tempo de resposta das consultas filtradas em até 3 segundos em condições normais;
- **B-08 (Compatibilidade Multiplataforma):** suporte a telas com largura mínima de 360 pixels e compatibilidade com os principais navegadores de mercado (Chrome, Firefox e Edge);
- **B-09 (Segurança e Autenticação):** armazenamento seguro de credenciais administrativas exclusivamente com hash criptográfico e proteção de rotas restritas.

Adicionalmente, os requisitos classificados como **SHOULD** (Tabelas 10 e 11) orientam o desenho modular da solução para viabilizar sua extensão:

- **B-06 (Comparação entre Estados):** contraste direto de indicadores entre duas unidades federativas;
- **B-10 (Tabelas e Gráficos Complementares):** apresentação dos indicadores em formatos tabulares e gráficos analíticos.

Por fim, os requisitos classificados como **COULD** (Tabelas 10 e 11) estabelecem funcionalidades suplementares:

- **B-07 (Linha do Tempo e Evolução Histórica):** acompanhamento temporal da variação dos índices;
- **B-11 (Exportação de Dados):** download dos dados filtrados em formatos abertos (CSV e JSON).

2 Representação Arquitetural

2.1 Definições

O Brasil Hub seguirá uma arquitetura Cliente-Servidor em Camadas (Three-Tier), composta por três níveis lógicos principais: apresentação, aplicação e persistência. A camada de

apresentação corresponde ao Front-end executado no navegador do usuário; a camada de aplicação corresponde ao Back-end desenvolvido em Python e à API RESTful implementada com FastAPI; e a camada de persistência corresponde ao banco de dados SQLite.

A separação estrita entre essas camadas tem como objetivo principal reduzir o acoplamento entre a interface de usuário, as regras de negócio, a validação de dados e os mecanismos de armazenamento físico, promovendo modularidade, manutenibilidade e testabilidade independente dos componentes (BASS; CLEMENTS; KAZMAN, 2021; SOMMERVILLE, 2018).

A comunicação entre a camada de apresentação (cliente) e a camada de aplicação (servidor) é estabelecida pelo protocolo HTTP/HTTPS, aderindo aos princípios arquiteturais do modelo REST (FIELDING, 2000). As requisições e respostas estruturadas utilizam a notação JSON, enquanto o envio de lotes de dados brutos por administradores é realizado por meio do padrão de codificação *multipart/form-data*.

A camada de apresentação não possuirá acesso direto ao banco de dados. Todas as consultas e alterações deverão passar pela camada de aplicação, permitindo centralizar regras de validação, autenticação, autorização e tratamento dos dados.

2.2 Justificativa da escolha

A seleção da arquitetura Cliente-Servidor em Camadas decorre diretamente dos requisitos e restrições estabelecidos no Documento de Visão e Projeto (GRUPO DIJKSTRA, 2026):

1. **Atendimento ao Desempenho (Requisito B-05):** O requisito B-05 estipula que as respostas às consultas de indicadores socioeconômicos devem ocorrer em até 3 segundos. A dissociação entre cliente e servidor viabiliza o emprego de Python e Pandas na camada de aplicação para execução de operações vetorizadas em memória e agregação de dados tabulares com alta performance, enquanto o FastAPI gerencia requisições

assíncronas concorrentes de baixa latência, retornando exclusivamente cargas úteis JSON otimizadas para o cliente.

2. **Segurança e Validação no Servidor (Requisitos B-04 e B-09):** O requisito B-04 exige a ingestão de arquivos CSV governamentais com garantia estrita de integridade estrutural, e o requisito B-09 impõe que credenciais administrativas nunca circulem ou residam desprotegidas. Manter a lógica de validação, parsing com Pandas e hashing de senhas isoladas na camada de aplicação impede que código malicioso ou arquivos corrompidos afetem diretamente o armazenamento, assegurando que o Front-end permaneça como uma casca de visualização sem privilégios de persistência.
3. **Compatibilidade Multiplataforma e Responsividade (Requisito B-08):** O requisito B-08 estabelece suporte pleno a dispositivos a partir de 360 pixels de largura e navegadores modernos (Chrome, Firefox, Edge). O desacoplamento permite que a interface cliente seja construída com tecnologias web puras (HTML5, CSS3, JavaScript e Leaflet), operando de maneira leve e nativa no motor de renderização do cliente sem a sobrecarga de renderização server-side, garantindo fluidez e rápida resposta visual em telas reduzidas.
4. **Alinhamento com a Estrutura da Equipe:** A segregação em camadas viabiliza o desenvolvimento paralelo entre as duplas de trabalho da equipe Dijkstra (frente de interface e frente de dados/API), minimizando dependências de código e conflitos de integração durante as Sprints de desenvolvimento.

A camada de persistência utiliza o SQLite, centralizando o armazenamento estruturado dos indicadores socioeconômicos e das credenciais administrativas. A escolha do SQLite fundamenta-se na sua simplicidade arquitetural, baixo consumo de recursos, portabilidade em arquivo único e perfeita integração com o ecossistema Python, permitindo desenvolvimento e testes ágeis sem a necessidade de gerenciamento de um serviço externo de banco de dados.

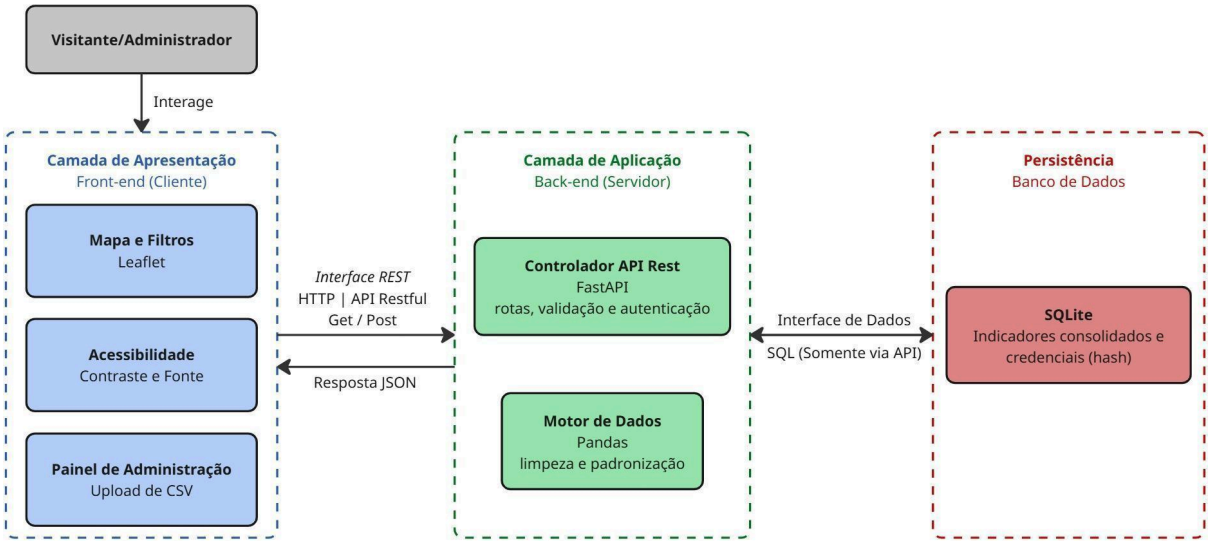
2.3 Detalhamento

A arquitetura é dividida em três camadas fundamentais :

1. **Interface do Usuário (Front-end)**: Camada responsável pela interação com o usuário final, exibição gráfica dos mapas e interface do Administrador para modificação de dados. Captura interações dos usuários e manda para a API.
2. **Servidor de API (Back-end):** Construído em FastAPI para tratar requisições e manipulação de Pandas, executar validações e processar as consultas feitas pelos usuários.
3. **Camada de Persistência (Banco de Dados):** Responsável pela integridade e persistência de dados das tabelas de indicadores socioeconômicos tratados e de usuários administradores com credenciais armazenadas com hash.

A disposição estrutural desses níveis e o fluxo de dados entre os componentes do sistema estão representados no diagrama arquitetural da **Figura 1**.

Figura 1 - Diagrama Arquitetural Genérico:



Fonte: *elaborado por Lucas Nery.*

2.3.1 Descrição do Diagrama Arquitetural

O sistema adota o estilo arquitetural em camadas, com comunicação cliente-servidor, dividido em três camadas: apresentação, aplicação e persistência. Cada camada tem responsabilidades bem definidas e se comunica apenas com a camada adjacente, por meio de interfaces explícitas. Isso garante separação de responsabilidades, segurança no acesso aos dados e facilidade de manutenção.

A camada de apresentação (front-end) é uma aplicação web executada no navegador e é o ponto de interação dos dois atores do sistema: o visitante, que consulta os indicadores, e o administrador, que também mantém os dados. Ela reúne três componentes: o mapa e filtros, implementado com Leaflet, que exibe os indicadores de forma georreferenciada e interativa; a acessibilidade, que permite ajustar contraste e tamanho de fonte; e o painel de administração, que permite ao administrador, após autenticação, enviar arquivos CSV para atualizar a base.

A camada de aplicação (back-end) é um servidor desenvolvido em Python que concentra as regras do sistema. O Controlador da API, implementado com FastAPI, expõe as rotas HTTP, valida as requisições e encaminha as operações para os serviços responsáveis. Durante a execução, a aplicação FastAPI será servida pelo Uvicorn, responsável pelo atendimento das requisições HTTP no ambiente configurado.

O Motor de Dados, implementado com Pandas, realiza a leitura, validação, limpeza, padronização e consolidação dos dados provenientes dos arquivos CSV. Para a verificação do comportamento da API, os testes automatizados poderão utilizar HTTPX em conjunto com Pytest, permitindo avaliar as respostas dos endpoints e a integração entre os componentes do back-end.

A camada de persistência utiliza um banco SQLite, que armazena os indicadores consolidados e as credenciais do administrador, estas em formato *hash*. Esse banco só é acessado pela camada de aplicação.

A comunicação entre as camadas ocorre por dois conectores. A Interface REST liga a apresentação à aplicação, usando HTTP com requisições GET (consulta de indicadores e filtros) e POST (login e envio de CSV), e respostas em JSON ou mensagens de erro. A Interface de Dados liga a camada de aplicação à persistência por meio das classes de repositório responsáveis pelo acesso ao SQLite. Dessa forma, o Controlador da API não

acessa diretamente o banco de dados, e a apresentação nunca realiza acesso direto à persistência.

Tabela 3 - Componentes da arquitetura:

Componente	Tecnologia	Responsabilidade	Entrada	Saída
Interface Web	JavaScript/Leaflet	Interação com usuários, mapas, filtros e acessibilidade	Ações do usuário	Requisições HTTP e apresentação dos resultados
Controlador da API	FastAPI	Receber requisições, validar dados e encaminhar operações	HTTP/JSON ou CSV	JSON, mensagens de erro ou resultado da operação
Serviço de Indicadores	Python	Coordenar consultas e filtros	Estado, dimensão e período	Registros filtrados
Serviço de Atualização	Python/Pandas	Coordenar atualização da base	Arquivo CSV	Resultado da validação e atualização
Módulo de Autenticação	Python	Validar Credenciais e autorização	Usuário e senha/token	Autorização ou rejeição
Processador de Dados	Pandas	Ler, validar, limpar e padronizar dados	CSV/DataFrame	DataFrame padronizado
Banco de Dados	SQLite	Persistir credenciais e indicadores	Consultas/alterações SQL	Registros persistidos

Fonte: *elaborado por Josef Wojtyla (2026).*

A arquitetura apresenta três camadas principais e módulos especializados distribuídos entre elas. A camada de apresentação concentra os componentes responsáveis pela interação com os usuários. A camada de aplicação concentra a API, a autenticação, os serviços de aplicação e o processamento dos dados. A camada de persistência concentra o armazenamento das informações.

A comunicação entre as camadas ocorre por interfaces explícitas. A interface REST utiliza HTTP para transportar requisições e respostas entre o navegador e a API. Requisições de consulta utilizam principalmente o método GET, enquanto operações que produzem alterações no sistema, como login e upload de arquivos, utilizam POST. As respostas estruturadas são representadas em JSON. O upload de arquivos CSV utiliza *multipart/form-data*, devido à necessidade de transportar o arquivo propriamente dito.

No interior do Back-end não existe uma comunicação de rede entre os módulos. A comunicação ocorre por chamadas de funções e importações entre módulos Python, seguindo a direção definida na arquitetura lógica. O Front-end depende exclusivamente da API, a API utiliza os serviços e os módulos de autenticação, os serviços de dados utilizam o processamento e o acesso à persistência, e o módulo de autenticação consulta as credenciais armazenadas no banco.

O banco de dados SQLite armazena dois grupos principais de informações: credenciais administrativas protegidas por hash e os indicadores socioeconômicos tratados. O Front-end nunca acessa diretamente o banco, garantindo que qualquer acesso aos dados seja intermediado pelo Back-end.

2.3.2 Modelo de persistência

O banco de dados do Brasil Hub utilizará SQLite. A base será responsável por armazenar os dados socioeconômicos tratados e as informações necessárias à autenticação dos administradores.

A estrutura lógica mínima da base deverá contemplar uma tabela de administradores e uma estrutura para armazenamento dos indicadores socioeconômicos.

A tabela de administradores deverá armazenar, no mínimo, um identificador, o nome de usuário e a senha protegida por hash. A senha original não deverá ser persistida.

A estrutura dos indicadores deverá manter, no mínimo, informações referentes ao indicador, estado ou unidade territorial, dimensão socioeconômica, valor, período de referência, unidade de medida e fonte dos dados, permitindo preservar o contexto necessário para exibição e comparação.

O acesso ao SQLite será encapsulado pela camada de persistência, evitando que os demais módulos dependam diretamente da implementação do banco. Essa decisão reduz o acoplamento e permite uma eventual substituição da tecnologia de persistência sem alterações relevantes no Front-end ou nas regras da aplicação.

2.4 Metas e restrições arquiteturais

As metas e restrições arquiteturais foram definidas a partir dos requisitos funcionais e não funcionais do Documento de Visão e têm como objetivo garantir desempenho, manutenibilidade, segurança, compatibilidade e consistência estrutural do sistema.

- **1. Comunicação por API REST:** a comunicação entre Front-end e Back-end deverá ocorrer exclusivamente por HTTP por meio da API REST. Requisições de consulta deverão utilizar GET, enquanto operações de criação ou alteração de dados deverão utilizar POST, respeitando a semântica dos métodos HTTP.
- **2. Formato de dados:** respostas estruturadas da API deverão utilizar JSON. O envio de arquivos CSV deverá utilizar *multipart/form-data*.
- **3. Desempenho:** as consultas realizadas por meio dos filtros deverão apresentar resposta da API em até 3 segundos, conforme estabelecido pelo requisito B-05. A medição deverá ser realizada em ambiente de teste controlado, utilizando o procedimento definido no roteiro de testes do projeto.
- **4. Compatibilidade:** a interface deverá funcionar em telas com largura mínima de 360 pixels e nos navegadores Chrome, Firefox e Edge, sem quebra de layout ou perda das funcionalidades classificadas como MUST.
- **5. Segurança:** o armazenamento de senhas administrativas deverá ser realizado exclusivamente utilizando valores de hash, impedindo o armazenamento da senha em

texto simples. O acesso às operações administrativas deverá ser condicionado à autenticação e autorização do usuário.

- **6. Padrão de desenvolvimento Back-end:** o código Python deverá seguir PEP 8, utilizando *snake_case* para funções e variáveis e *PascalCase* para classes.
- **7. Padrão de desenvolvimento Front-end:** o código JavaScript deverá seguir as regras de lintagem definidas pelo ESLint, utilizando *camelCase* para funções e variáveis e *UPPER_SNAKE_CASE* para constantes.
- **8. Controle de versão, revisão de código e Integração Contínua:** as alterações deverão ser desenvolvidas em branches de trabalho e submetidas a Pull Requests para revisão antes da integração à branch de destino definida pela política de branches do projeto. O GitHub Actions será utilizado para executar automaticamente os testes do back-end nos eventos configurados, incluindo a abertura e a atualização de Pull Requests. Os testes deverão ser executados em um ambiente reproduzível, com as dependências necessárias instaladas a partir dos arquivos versionados no repositório. Os resultados da execução deverão permanecer disponíveis no GitHub Actions. Quando os testes falharem, a equipe deverá analisar os registros, corrigir os problemas identificados e executar novamente as verificações antes de considerar a alteração validada. A automação não substitui a revisão humana do código nem os demais testes previstos no projeto.

2.5 Visões

As diferentes visões arquiteturais apresentam o Brasil Hub sob perspectivas complementares, permitindo compreender o sistema quanto à sua utilização pelos atores, à organização lógica dos módulos, à estrutura interna dos componentes e ao ambiente no qual a aplicação será implantada. Juntas, essas visões complementam a representação arquitetural apresentada nas seções anteriores e permitem relacionar os requisitos do produto às decisões técnicas adotadas.

2.5.1 Visão uso

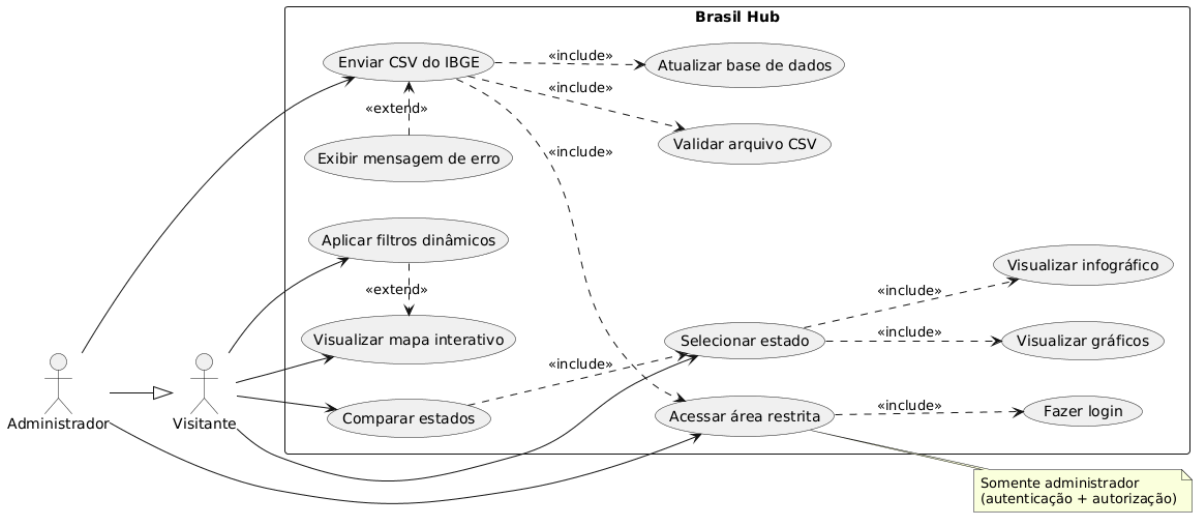
A visão de uso apresenta o sistema a partir das funcionalidades disponíveis para os atores que interagem com o Brasil Hub. São definidos dois atores principais: o Visitante e o Administrador.

O Visitante não necessita de autenticação e pode consultar os indicadores socioeconômicos disponibilizados pelo sistema, utilizar o mapa interativo, aplicar os filtros por estado e pelas dimensões do IDH, utilizar os recursos de acessibilidade e, quando implementados, utilizar os recursos complementares de comparação, tabelas, linha do tempo e exportação de dados.

O Administrador representa o usuário autenticado responsável pela manutenção dos dados da plataforma. Além das funcionalidades disponíveis ao Visitante, possui acesso à área administrativa e pode realizar o envio de arquivos CSV do IBGE para atualização da base de dados.

A **Figura 2** apresenta o diagrama de casos de uso do sistema, destacando os casos de uso prioritários do produto — consulta aos indicadores no mapa interativo (B-01), aplicação de filtros dinâmicos (B-02), utilização dos recursos de acessibilidade (B-03) e gerenciamento administrativo por meio de upload de dados (B-04 e B-09) — evidenciando a relação de herança na qual o Administrador estende as capacidades do Visitante.

Figura 2 - Diagrama de Caso de Uso

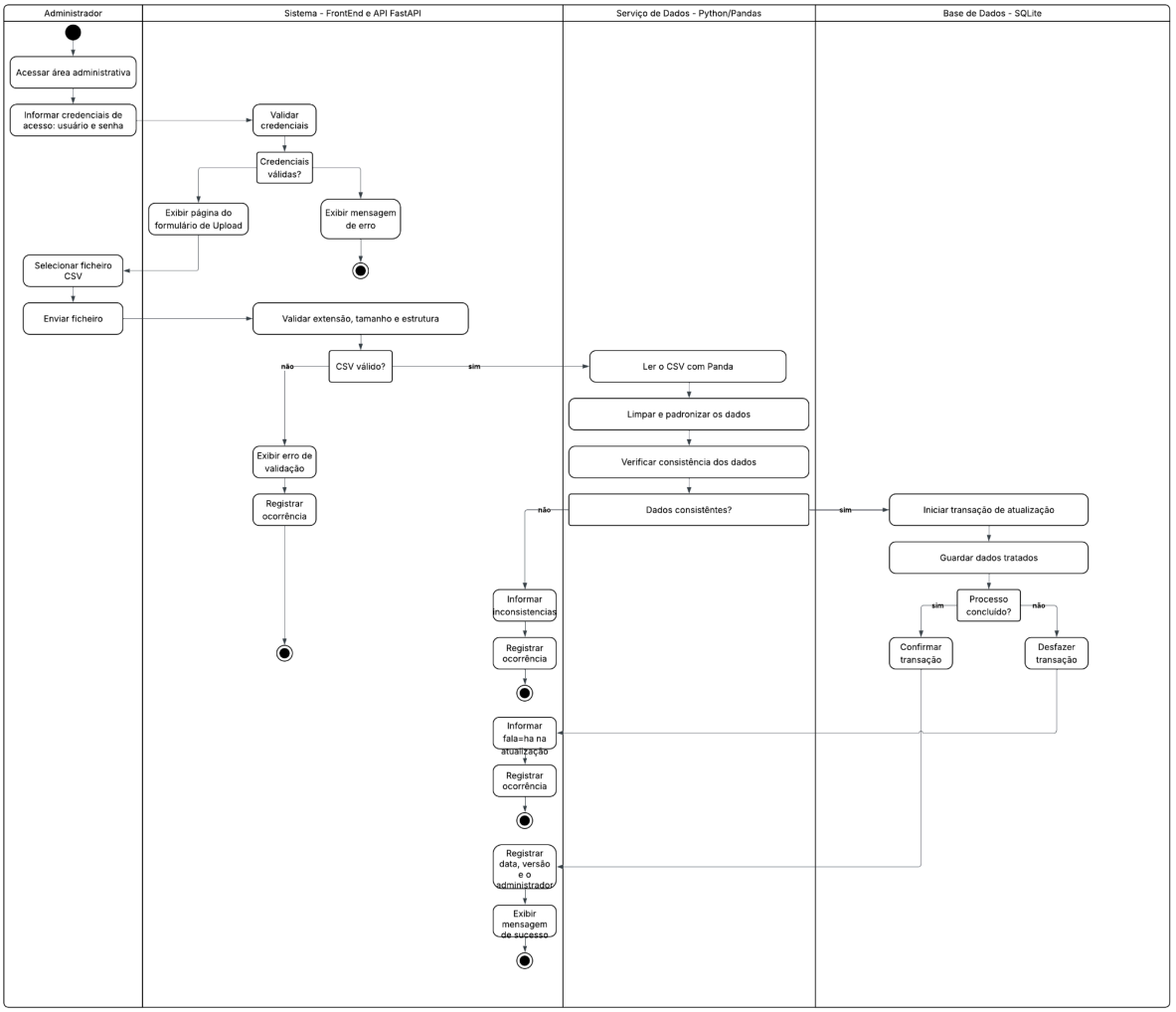


Fonte: *produzido por Felipe George*

A funcionalidade selecionada para o diagrama de atividades é o processo de atualização da base de dados pelo Administrador, por apresentar maior quantidade de etapas e dependências entre os componentes arquiteturais. O processo envolve autenticação e autorização prévia, recepção do arquivo CSV, validação estrutural do formato, processamento analítico com a biblioteca Pandas, persistência atômica no banco de dados e notificação de sucesso ou falha ao usuário.

Esse fluxo operacional é detalhado na **Figura 3**, que apresenta o diagrama de atividades da área administrativa.

Figura 3 - Diagrama de Atividades, funcionalidade de Painel do Administrador



Fonte: *produzido por Fabiana Reis.*

A visão de uso também evidencia decisões que influenciaram a escolha do estilo arquitetural. Os requisitos de tratamento e padronização dos arquivos, desempenho da API, segurança da área administrativa e separação entre interface e processamento justificam a utilização da arquitetura Cliente-Servidor em Camadas. Dessa maneira, o navegador concentra a apresentação dos dados, enquanto o servidor concentra o processamento, as regras de acesso e a persistência.

2.5.2 Visão de organização lógica

O Brasil Hub é organizado logicamente em seis módulos principais, distribuídos entre as camadas da arquitetura. A divisão foi definida para concentrar responsabilidades relacionadas e minimizar dependências desnecessárias entre os componentes.

O módulo *src/frontend* concentra a apresentação e a interação com o usuário. O módulo *src/api* representa a fronteira de comunicação entre o navegador e o Back-end. O módulo *src/services* concentra as regras e a coordenação das operações da aplicação. O módulo *src/data* concentra a ingestão, validação, transformação e disponibilização dos dados. O módulo *src/auth* concentra as responsabilidades de autenticação e autorização e atua como módulo transversal de segurança. Por fim, *src/database* encapsula o acesso ao SQLite.

As dependências seguem uma direção única, reduzindo o acoplamento entre os módulos. O *frontend* utiliza a *api*; a *api* utiliza *services* e *auth*; *services* utiliza *data*; *data* utiliza *database*; e *auth* utiliza *database* para consulta das credenciais administrativas.

Dessa forma, o Front-end não possui conhecimento da estrutura interna do banco de dados, e os módulos de apresentação não implementam regras de persistência diretamente.

Tabela 4 - Tabela de Módulos

Camada	Módulo	Papel resumido
Interface	src/frontend	Dashboard, mapas, filtros, acessibilidade e área administrativa
API	src/api	Rotas HTTP e contratos de entrada e saída
Lógica de aplicação	src/services	Coordenação das consultas e da atualização da base

Camada	Módulo	Papel resumido
Segurança	src/auth	Autenticação, autorização e proteção das senhas
Dados	src/data	Leitura, validação, tratamento e acesso aos indicadores
Persistência	src/database	Acesso às credenciais administrativas no SQLite

Fonte: *produzido por Ágatha Natasha*

2.5.2.1 Razão lógica de cada módulo:

- **src/frontend (JavaScript e Leaflet).** Concentra tudo o que o usuário vê e manipula. Separa as páginas (pages/) dos componentes reutilizáveis de mapa e filtros (components/), dos recursos de acessibilidade de fonte e contraste (accessibility/), do envio de requisições (api_client/) e dos estilos e recursos (assets/). Assim, a aparência pode mudar sem afetar as regras do sistema, e o acesso à API fica em um único ponto.
- **src/api (FastAPI).** É a fronteira do back-end: recebe as requisições do navegador e devolve as respostas. Cada rota tem uma responsabilidade: consultas de indicadores (indicators.py), upload de CSV pela administração (admin.py) e login (auth.py). Os schemas/ definem os contratos de entrada e saída, de modo que a validação do formato fica na borda do sistema e a lógica interna não depende dela.
- **src/services.** Fica entre as rotas e o tratamento de dados. indicators.py coordena as consultas e os filtros por estado e dimensões do IDH/IDHM, e updates.py coordena a atualização da base. Com isso, as rotas apenas delegam e as regras de negócio podem ser alteradas ou testadas sem depender do protocolo HTTP.
- **src/data (Pandas).** Reúne o ciclo de vida dos indicadores: ingestion.py lê o CSV, validation.py verifica estrutura e valores, processing.py faz limpeza e padronização, e storage.py dá acesso aos indicadores já tratados. Isolar esse ciclo mantém a qualidade dos dados em um só lugar e evita que dados inválidos cheguem ao dashboard.
- **src/auth.** Protege a área administrativa. authentication.py confirma a identidade de quem faz login, permissions.py decide se o usuário pode executar a ação (por exemplo, atualizar a base) e password_hash.py garante que as senhas sejam armazenadas protegidas. Mantê-lo separado concentra as

regras de segurança, que podem ser revisadas e evoluídas sem alterar o restante.

- **src/database (SQLite).** Encapsula o acesso ao banco que guarda apenas as credenciais administrativas. connection.py gerencia a conexão e admin_repository.py consulta as credenciais. Apenas este módulo conhece o SQLite; se o banco for trocado no futuro, só ele precisa mudar.

2.5.2.2 Comunicação entre os módulos:

As dependências seguem uma direção única, reduzindo o acoplamento entre os módulos. O frontend utiliza a api; a api utiliza services e auth; services utiliza data; data utiliza database; e auth utiliza database para consulta das credenciais administrativas.

2.5.2.3 Fluxos principais

A organização lógica pode ser compreendida a partir de dois fluxos principais.

No fluxo de consulta, o Visitante interage com a interface web e seleciona os critérios desejados. O Front-end envia uma requisição HTTP para a API, que valida os parâmetros e encaminha a operação para o serviço responsável. O serviço consulta os dados tratados por meio da camada de dados e da persistência, retornando à API os registros correspondentes. A API transforma o resultado em JSON e o envia ao Front-end, que atualiza a visualização do mapa, tabela ou gráfico.

No fluxo de atualização, o Administrador realiza a autenticação e, após autorização, envia um arquivo CSV pela área administrativa. A API recebe o arquivo, o encaminha aos serviços responsáveis e o módulo de dados realiza a leitura, validação, limpeza e padronização utilizando Pandas. Após a validação, os dados são persistidos no SQLite e o resultado da operação é retornado ao Administrador.

Essa divisão permite que a lógica de apresentação, as regras da aplicação, o processamento dos dados, a segurança e a persistência permaneçam isolados, facilitando a manutenção e a realização de testes.

Tabela 5 - Tabela de Comunicação entre os Módulos

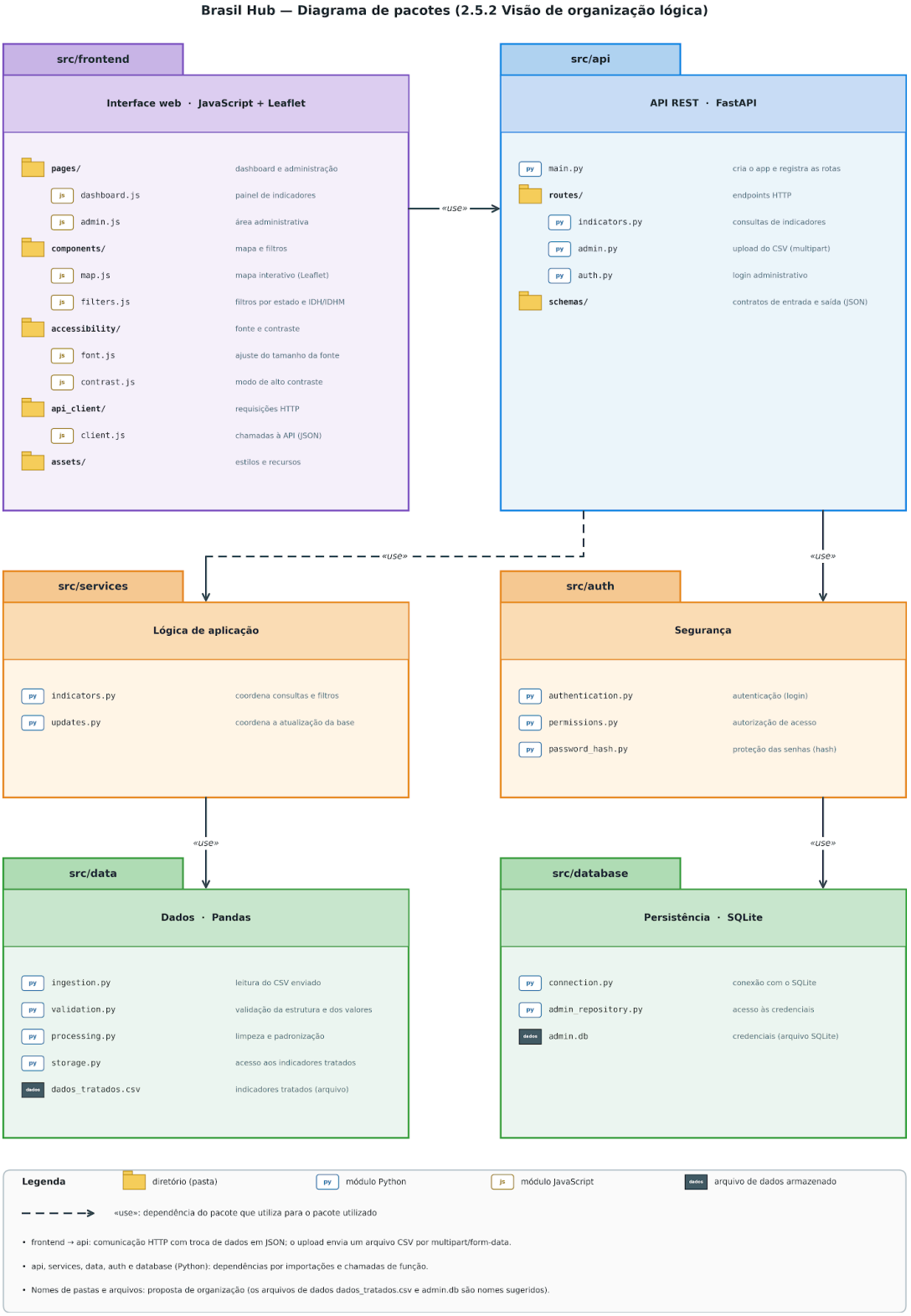
Origem → Destino	Forma de comunicação	Interface
frontend → api	Requisições HTTP feitas por api_client/, com dados em JSON	Rotas HTTP de consulta de indicadores, login e atualização da base
frontend → api (upload)	Envio do arquivo CSV por multipart/form-data, pela área administrativa	Rota de upload em routes/admin.py
api → services	Importação de módulos e chamada de funções Python	Funções de indicators.py e updates.py
api → auth	Importação de módulos e chamada de funções Python	Funções de autenticação e autorização
services → data	Importação de módulos e chamada de funções Python	Funções de ingestão, validação, tratamento e acesso aos indicadores
auth → database	Importação de módulos e chamada de funções Python	Funções de admin_repository.py para consultar credenciais
data → database	Importação de módulos e chamadas de funções Python	Funções de persistência dos indicadores e conexão com SQLite

Fonte: *produzido por Ágatha Natasha*

Em resumo, a comunicação entre o navegador e o servidor usa HTTP com JSON, e o CSV do upload segue por multipart/form-data. Dentro do back-end, os módulos Python se comunicam por importações e chamadas de função, sem protocolo de rede. Os dois fluxos principais são:

- **Consulta de indicadores:** frontend → api → services → data → database.
- **Atualização da base:** o administrador faz login (api → auth → database) e envia o CSV, que passa por api → services → data → database , envolvendo as etapas de leitura, validação, tratamento e persistência.

Figura 4 - Diagrama de pacotes



Fonte: *produzido por Ágatha Natasha*

2.5.3 Visão estrutural

A visão estrutural apresenta os principais elementos internos do Brasil Hub e as relações entre eles, complementando a visão lógica apresentada anteriormente. Enquanto a visão lógica organiza os módulos por responsabilidade, a visão estrutural apresenta os componentes que participam diretamente da execução do sistema e as relações existentes entre eles.

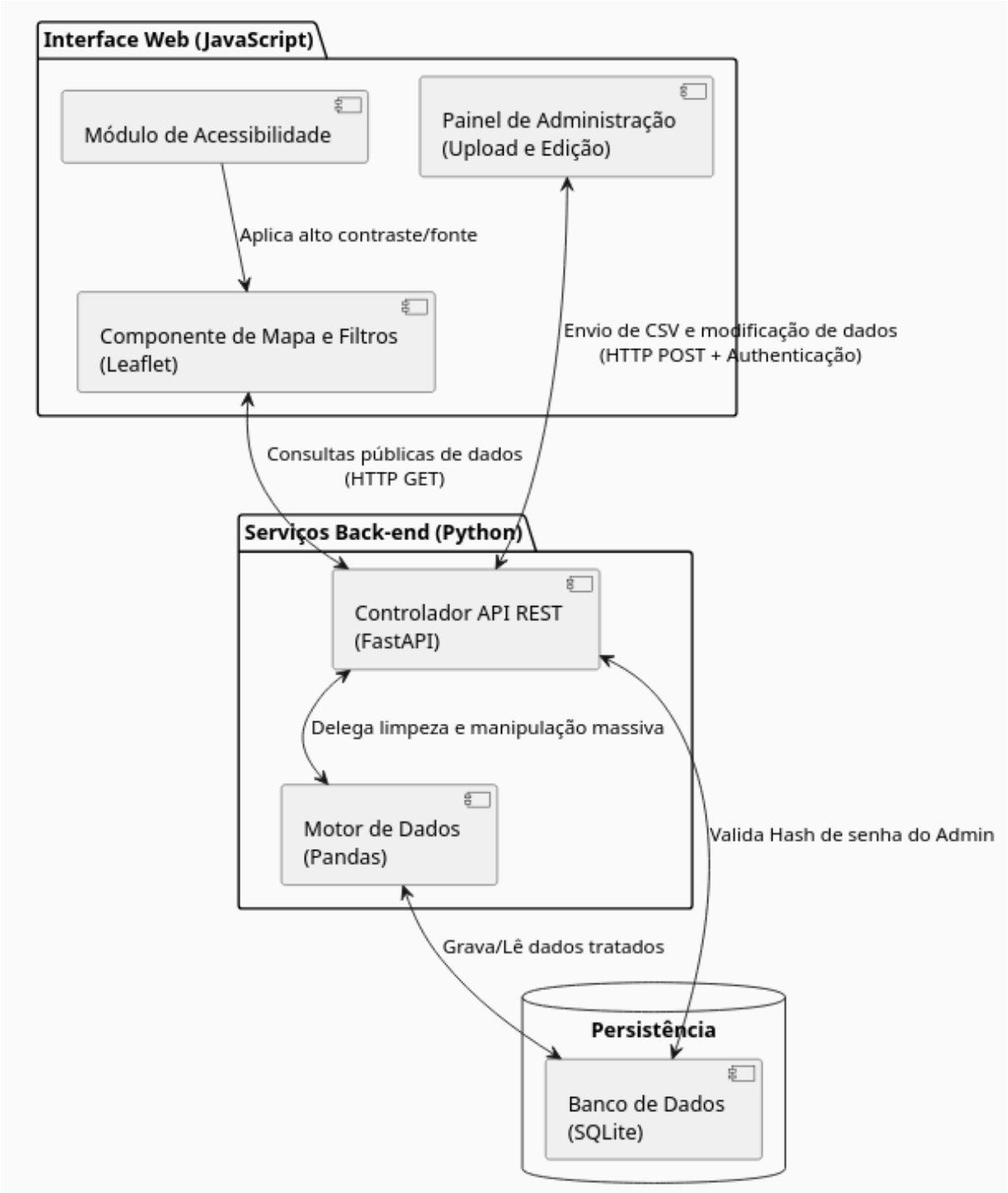
Na camada de apresentação estão os componentes responsáveis pela interface web, incluindo o mapa interativo, os filtros, os recursos de acessibilidade e o painel administrativo. Esses componentes utilizam a API como única interface de comunicação com o Back-end.

Na camada de aplicação, o Controlador da API recebe as requisições, valida os parâmetros e encaminha as operações para os serviços responsáveis. O Processador de Dados utiliza Pandas para executar as etapas de leitura, validação, limpeza e padronização dos dados. O módulo de autenticação valida as credenciais e verifica as permissões necessárias para as operações administrativas.

Na camada de persistência, o banco de dados SQLite é acessado por meio das classes IndicadorRepository e AdministradorRepository, responsáveis por encapsular as operações de persistência das entidades correspondentes. O IndicadorRepository trata da persistência dos indicadores socioeconômicos, enquanto o AdministradorRepository realiza as operações relacionadas às credenciais administrativas. Essa separação impede que os componentes de apresentação ou os atores externos acessem diretamente o banco de dados e reduz o acoplamento entre a aplicação e a tecnologia de persistência.

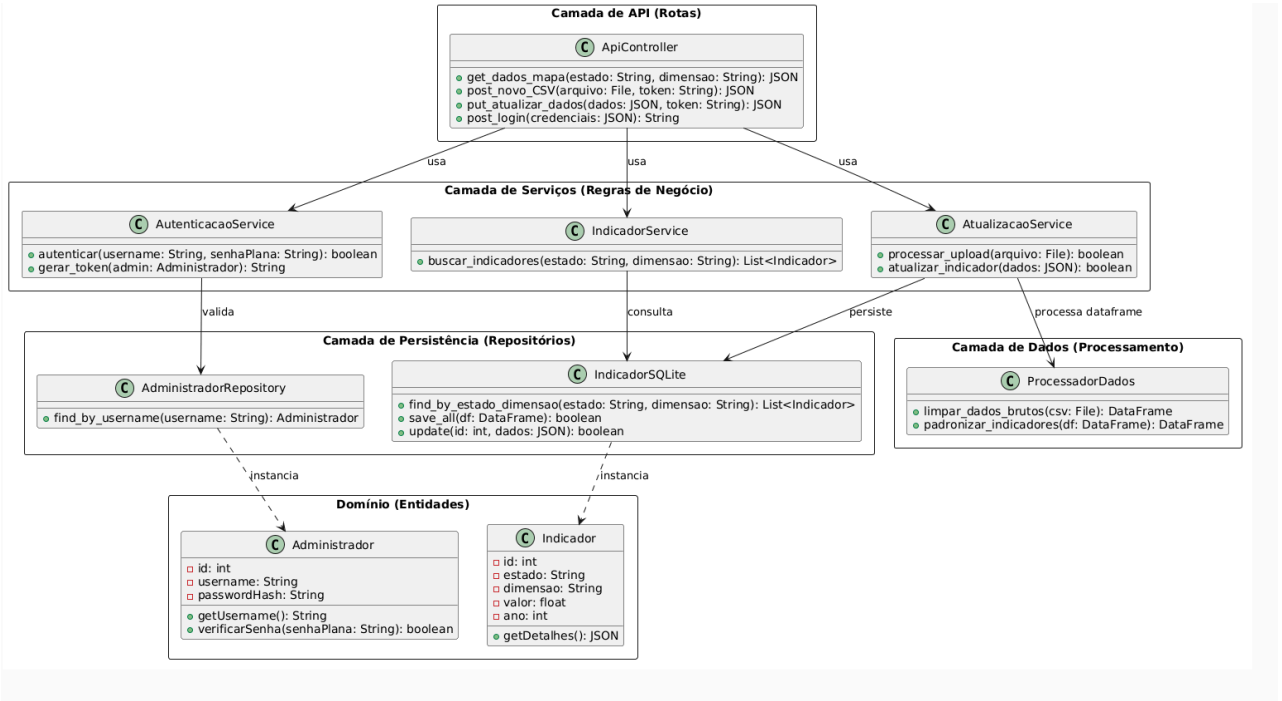
O diagrama de componentes apresenta as dependências entre os principais componentes de execução. O diagrama de classes apresenta as entidades e serviços necessários para representar as responsabilidades estruturais do sistema, devendo manter a separação entre atores externos, componentes de aplicação e entidades persistentes.

Figura 5 - Diagrama de componentes



Fonte: *elaborado por César Augusto.*

Figura 6 - Diagrama de Classes



Fonte: *elaborado por César Augusto.*

2.6 Visão de Implantação

O Brasil Hub é implantado em uma infraestrutura de computação em nuvem utilizando a Vercel (VERCEL, 2026) como plataforma de hospedagem da aplicação web e da API. A Vercel atua na disponibilização do Front-end (arquivos estáticos servidos via rede de distribuição de conteúdo — CDN) e na execução do Back-end Python por meio de Functions, processando sob demanda as requisições direcionadas à API REST.

A camada de persistência utiliza o banco de dados relacional **SQLite**, armazenando em arquivo estruturado os dados socioeconômicos tratados e as credenciais administrativas protegidas por hash. A escolha do SQLite fundamenta-se na sua simplicidade operacional, dispensa de provisionamento de servidor externo de banco de dados e facilidade de portabilidade entre os membros da equipe.

Durante o ciclo de desenvolvimento, homologação e execução dos testes automatizados, o ambiente do projeto é padronizado por meio de contêineres Docker (DOCKER, 2026). O arquivo *docker-compose.yml* unifica a versão da imagem do Python e as dependências do FastAPI e do Pandas, assegurando que

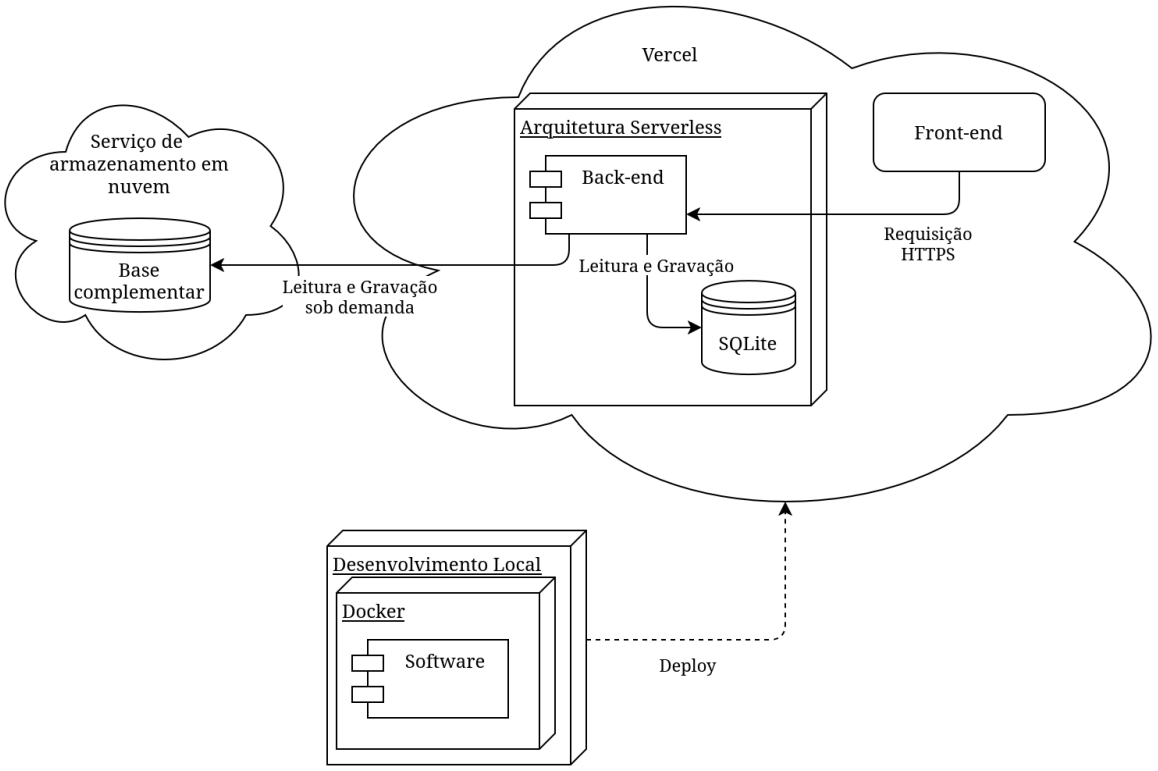
todos os membros da equipe e os executores do GitHub Actions trabalhem em ambiente idêntico, mitigando inconsistências decorrentes de variações de sistemas operacionais.

O fluxo de entrega contínua obedece à política de branches descrita no Documento de Visão e Projeto (GRUPO DIJKSTRA, 2026):

1. As implementações são desenvolvidas em branches específicas de funcionalidade (*feature/*);
2. Mediante abertura de Pull Request e aprovação por revisão de pares, os códigos são integrados à branch de integração *developer*;
3. Ao término de cada Sprint e consolidação das entregas de valor, a branch *developer* é promovida para a branch *main*, na qual as entregas são formalizadas com a aposição de tags semânticas de release (*R1*, *R2* e *R3*), acionando o pipeline automatizado de compilação, teste e publicação contínua (*Continuous Deployment*) na Vercel e no GitHub Pages para a documentação técnica (MkDocs).

A disposição física dos nós de computação, serviços em nuvem e canais de comunicação entre as camadas está esquematizada na **Figura 7**, que apresenta o diagrama de implantação do sistema.

Figura 7 - Diagrama de Implantação



Fonte: *elaborado por Eduardo de Sousa.*

2.7 Restrições adicionais

Além das metas e restrições apresentadas na Seção 2.4, o sistema possui restrições relacionadas à segurança, usabilidade, compatibilidade, portabilidade e manutenção.

Quanto à segurança, a área administrativa deverá ser acessível somente após autenticação. O perfil Visitante não possuirá permissão para realizar alterações nos dados. O Administrador possuirá permissão adicional para enviar arquivos CSV e atualizar a base. As credenciais administrativas não deverão ser armazenadas em texto simples, devendo permanecer protegidas por mecanismo de hash.

Quanto à integridade dos dados, arquivos enviados pelo Administrador deverão passar pelas etapas de validação, tratamento e padronização antes de serem persistidos. Arquivos que não atendam ao formato ou às regras de dados esperadas deverão ser rejeitados, evitando que informações inválidas sejam incorporadas à base utilizada pelo dashboard.

Quanto à usabilidade e acessibilidade, a interface deverá disponibilizar os recursos de ajuste de tamanho de fonte, modo noturno e alto contraste definidos no requisito B-03. Esses recursos não deverão impedir o acesso às informações ou aos controles essenciais da aplicação.

Quanto à compatibilidade, a aplicação deverá permanecer utilizável em telas a partir de 360 pixels de largura e nos navegadores Chrome, Firefox e Edge, conforme o requisito B-08.

Quanto à portabilidade, o ambiente de desenvolvimento deverá ser reproduzível por meio de Docker, permitindo a configuração uniforme das dependências e reduzindo diferenças entre os ambientes utilizados pelos integrantes.

Quanto à manutenibilidade, a separação entre Front-end, API, serviços, autenticação, processamento de dados e persistência deverá ser preservada. Alterações em uma camada deverão causar o mínimo possível de impacto nas demais, respeitando as interfaces estabelecidas pela arquitetura.

3 Bibliografia

FASTAPI. **FastAPI documentation**. Disponível em: https://fastapi.tiangolo.com/. Acesso em: 7 out. 2026.

IBGE. **Instituto Brasileiro de Geografia e Estatística**. Disponível em: https://www.ibge.gov.br. Acesso em: 7 out. 2026.

LEAFLET. **Leaflet documentation**. Disponível em: https://leafletjs.com/reference.html. Acesso em: 7 out. 2026.

PANDAS. **pandas documentation**. Disponível em: https://pandas.pydata.org/docs/. Acesso em: 7 out. 2026.

SQLITE. **SQLite documentation**. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 7 out. 2026.

VERCEL. **Vercel Functions**. Disponível em: https://vercel.com/docs/functions. Acesso em: 7 out. 2026.